- **Name:** Razel Navales
- **Term:** 3

Machine Problem 005: Time Response of First-Order Systems

The Python Control Systems Library provides tools to simulate and analyze the time response of control systems. In this notebook, we use it to study the **step response** of first-order systems — including how to identify poles and zeros, compute the time-domain response, and measure key performance specifications.

Consult the Python Control Systems Documentation for more details.

Installation

The Python Control Systems Library is not a standard part of most Python distributions. Run the following commands once to install the required packages.

To install both the Control Systems library and Slycot in an existing conda environment, run: `!conda install -c conda-forge control slycot`

In [1]:
```
!conda install -c conda-forge slycot
!pip install control
!pip install seaborn
```

```
^C
Requirement already satisfied: control in C:\Users\Jheboy\anaconda3\envs\control-sys
tem-cpe2a\Lib\site-packages (0.10.2)
Requirement already satisfied: numpy>=1.23 in C:\Users\Jheboy\anaconda3\envs\control
-system-cpe2a\Lib\site-packages (from control) (2.4.4)
Requirement already satisfied: scipy>=1.8 in C:\Users\Jheboy\anaconda3\envs\control-
system-cpe2a\Lib\site-packages (from control) (1.17.1)
Requirement already satisfied: matplotlib>=3.6 in C:\Users\Jheboy\anaconda3\envs\con
trol-system-cpe2a\Lib\site-packages (from control) (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\Jheboy\anaconda3\envs\co
ntrol-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\Jheboy\anaconda3\envs\contro
l-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\Jheboy\anaconda3\envs\c
ontrol-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\Jheboy\anaconda3\envs\c
ontrol-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (1.5.0)
Requirement already satisfied: packaging>=20.0 in C:\Users\Jheboy\anaconda3\envs\con
trol-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (26.0)
Requirement already satisfied: pillow>=8 in C:\Users\Jheboy\anaconda3\envs\control-s
ystem-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (12.2.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\Jheboy\anaconda3\envs\contro
l-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\Jheboy\anaconda3\env
s\control-system-cpe2a\Lib\site-packages (from matplotlib>=3.6->control) (2.9.0.post
0)
Requirement already satisfied: six>=1.5 in C:\Users\Jheboy\anaconda3\envs\control-sy
stem-cpe2a\Lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.6->control)
(1.17.0)
Requirement already satisfied: seaborn in C:\Users\Jheboy\anaconda3\envs\control-sys
tem-cpe2a\Lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in C:\Users\Jheboy\anaconda3\env
s\control-system-cpe2a\Lib\site-packages (from seaborn) (2.4.4)
Requirement already satisfied: pandas>=1.2 in C:\Users\Jheboy\anaconda3\envs\control
-system-cpe2a\Lib\site-packages (from seaborn) (3.0.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in C:\Users\Jheboy\anaconda3
\envs\control-system-cpe2a\Lib\site-packages (from seaborn) (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in C:\Users\Jheboy\anaconda3\envs\co
ntrol-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in C:\Users\Jheboy\anaconda3\envs\contro
l-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in C:\Users\Jheboy\anaconda3\envs\c
ontrol-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.62.
1)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\Jheboy\anaconda3\envs\c
ontrol-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.
0)
Requirement already satisfied: packaging>=20.0 in C:\Users\Jheboy\anaconda3\envs\con
trol-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in C:\Users\Jheboy\anaconda3\envs\control-s
ystem-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.2.0)
Requirement already satisfied: pyparsing>=3 in C:\Users\Jheboy\anaconda3\envs\contro
l-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in C:\Users\Jheboy\anaconda3\env
s\control-system-cpe2a\Lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.
9.0.post0)
```

```
Requirement already satisfied: tzdata in C:\Users\Jheboy\anaconda3\envs\control-syst
em-cpe2a\Lib\site-packages (from pandas>=1.2->seaborn) (2026.2)
Requirement already satisfied: six>=1.5 in C:\Users\Jheboy\anaconda3\envs\control-sy
stem-cpe2a\Lib\site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->se
aborn) (1.17.0)
```

Library Usage

The control systems library is designed to work with a simplified syntax where libraries are imported without the standard prefixes.

In [2]:
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import control.matlab as control
```

Demonstration

Poles and Zeros of a Transfer Function

Given a transfer function $G(s)$:

- **Poles** are values of s that make $G(s) \to \infty$ (roots of the **denominator**)
- **Zeros** are values of s that make $G(s) = 0$ (roots of the **numerator**)

Consider the transfer function from **Example 1** of the lecture:

$$G(s) = \frac{s+2}{s+5}$$

This system has:

- One **zero** at $s = -2$
- One **pole** at $s = -5$

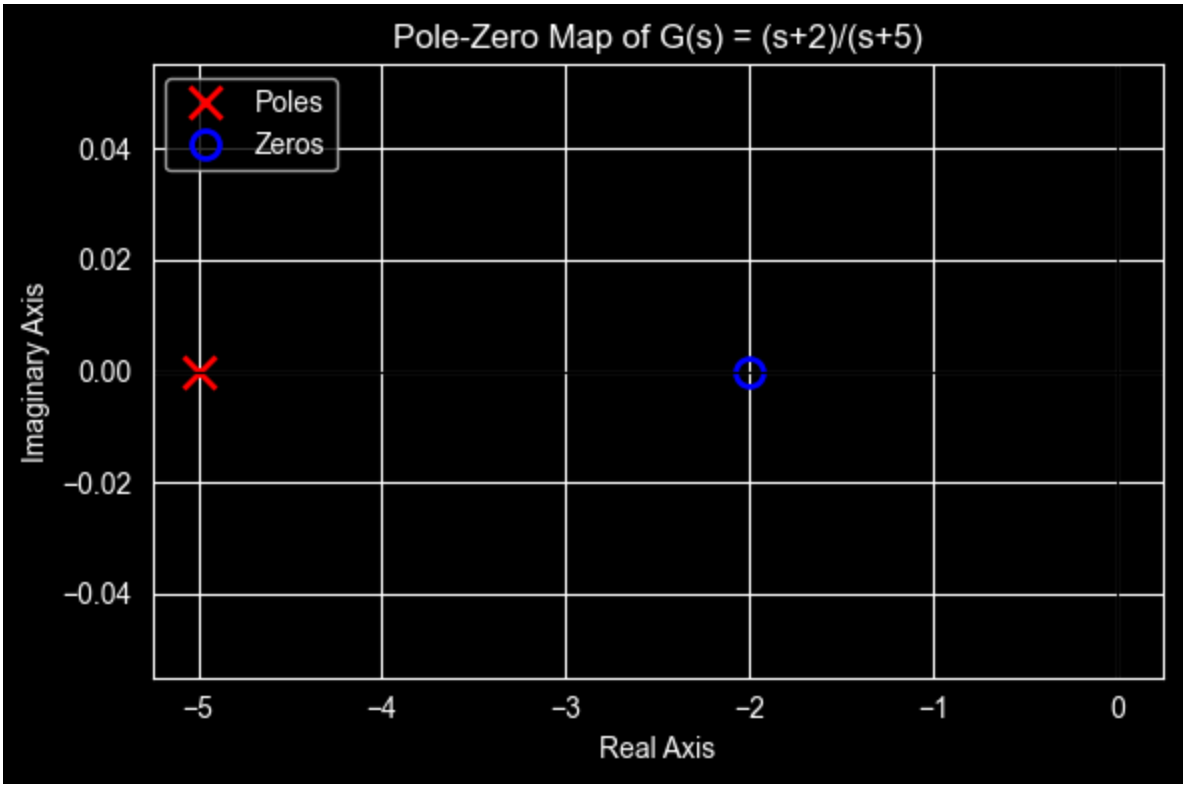
We can compute and display these using the control library.

In [3]:
```
G1 = control.tf([1, 2], [1, 5])
print('Transfer Function G(s):')
print(G1)

poles = control.pole(G1)
zeros = control.zero(G1)

print(f'Poles: {poles}')
print(f'Zeros: {zeros}')
```

```
Transfer Function G(s):
<TransferFunction>: sys[0]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

  s + 2
  -----
  s + 5
Poles: [-5.+0.j]
Zeros: [-2.+0.j]
```

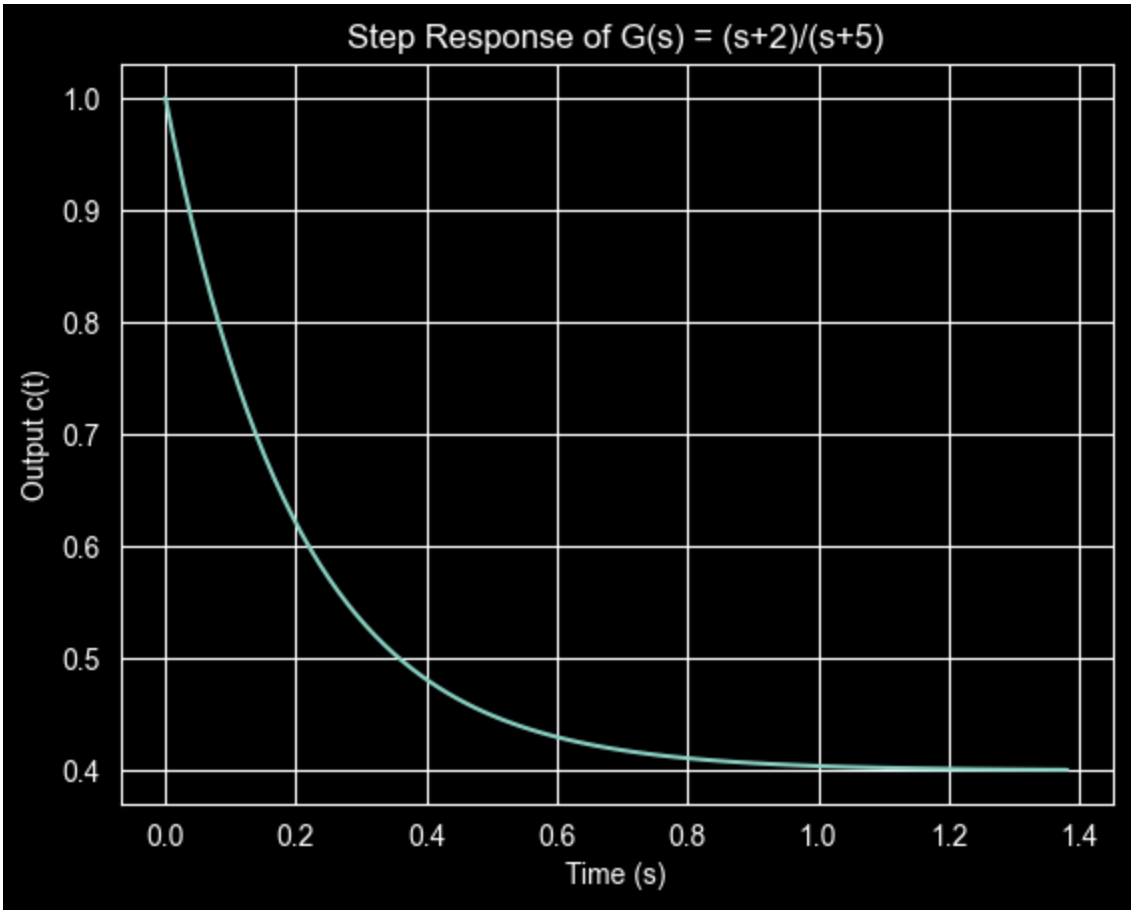
We can visualize the pole-zero locations on the complex s-plane using a **pole-zero map**. Poles are marked with **×** and zeros with ○.

In [4]:
```
fig, ax = plt.subplots(figsize=(6, 4))

# Plot poles and zeros
ax.plot(poles.real, poles.imag, 'x', markersize=12, markeredgewidth=2,
        color='red', label='Poles')
ax.plot(zeros.real, zeros.imag, 'o', markersize=10, markerfacecolor='none',
        markeredgewidth=2, color='blue', label='Zeros')

# Draw axes
ax.axhline(0, color='black', linewidth=0.8)
ax.axvline(0, color='black', linewidth=0.8)

ax.set_xlabel('Real Axis')
ax.set_ylabel('Imaginary Axis')
ax.set_title('Pole-Zero Map of G(s) = (s+2)/(s+5)')
ax.legend()
ax.grid(True)
plt.tight_layout()
plt.show()
```



The step response of this system combines contributions from both the forced response (due to the input) and the natural response (due to the pole). We compute it using `control.step()`.

In [5]:
```
y, t = control.step(G1)

plt.figure()
plt.plot(t, y)
plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title('Step Response of G(s) = (s+2)/(s+5)')
plt.grid(True)
plt.show()
```



First-Order System Transfer Function

A **first-order system** has the general transfer function:

$$G(s) = \frac{a}{s + a}$$

This is also written in **time-constant form** as:

$$G(s) = \frac{K}{\tau s + 1}$$

where $\tau = \frac{1}{a}$ is the **time constant** and K is the DC gain.

The system has one pole at $s = -a$ and **no finite zeros**.

For a **unit step input** $U(s) = \frac{1}{s}$, the output in the time domain is:

$$c(t) = K\left(1 - e^{-at}\right) = K\left(1 - e^{-t/\tau}\right)$$

Consider an example with $a = 3$, so $G(s) = \frac{3}{s + 3}$ and $\tau = \frac{1}{3}$ s.

In [6]:
```
a = 3          # pole location
tau = 1 / a    # time constant
```

```
G2 = control.tf([a], [1, a])
print('First-Order Transfer Function G(s):')
print(G2)
print(f'Pole at s = {-a}')
print(f'Time Constant τ = {tau:.4f} s')
```

```
First-Order Transfer Function G(s):
<TransferFunction>: sys[3]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

    3
  -----
  s + 3
Pole at s = -3
Time Constant τ = 0.3333 s
```

Step Response and Time Specifications

For a first-order system, three key **transient response specifications** are defined:

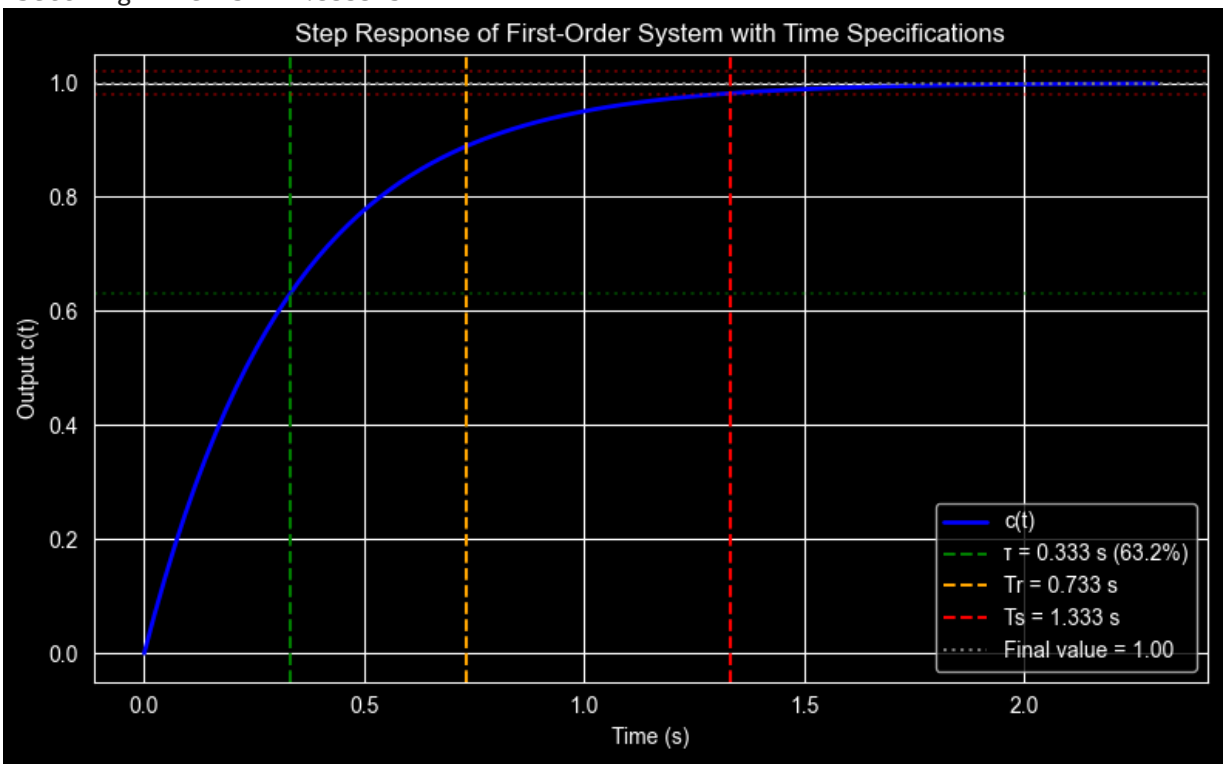
Specification	Symbol	Formula	Description
Time Constant	τ	$\tau = 1/a$	Time to reach 63.2% of the final value
Rise Time	T_r	$T_r \approx 2.2\tau$	Time to go from 10% to 90% of the final value
Settling Time	T_s	$T_s \approx 4\tau$	Time to remain within 2% of the final value

We compute and annotate these on the step response plot below.

In [7]:
```
# Compute time specifications
tau = 1 / a
Tr  = 2.2 * tau   # Rise time  (10% to 90%)
Ts  = 4.0 * tau   # Settling time (2% criterion)

print(f'Time Constant τ = {tau:.4f} s')
print(f'Rise Time    Tr = {Tr:.4f} s')
print(f'Settling Time Ts = {Ts:.4f} s')

# Step response
y, t = control.step(G2)
c_final = y[-1]   # steady-state value
```

```
plt.figure(figsize=(8, 5))
plt.plot(t, y, 'b-', linewidth=2, label='c(t)')

# Annotate time constant (63.2%)
plt.axvline(tau, color='green', linestyle='--', label=f'τ = {tau:.3f} s (63.2%)')
plt.axhline(0.632 * c_final, color='green', linestyle=':', alpha=0.5)

# Annotate rise time (10%–90%)
plt.axvline(Tr, color='orange', linestyle='--', label=f'Tr = {Tr:.3f} s')

# Annotate settling time (2% band)
plt.axvline(Ts, color='red', linestyle='--', label=f'Ts = {Ts:.3f} s')
plt.axhline(0.98 * c_final, color='red', linestyle=':', alpha=0.4)
plt.axhline(1.02 * c_final, color='red', linestyle=':', alpha=0.4)

# Final value
plt.axhline(c_final, color='gray', linestyle=':', label=f'Final value = {c_final:.2

plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title('Step Response of First-Order System with Time Specifications')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Time Constant τ = 0.3333 s
Rise Time    Tr = 0.7333 s
Settling Time Ts = 1.3333 s
```

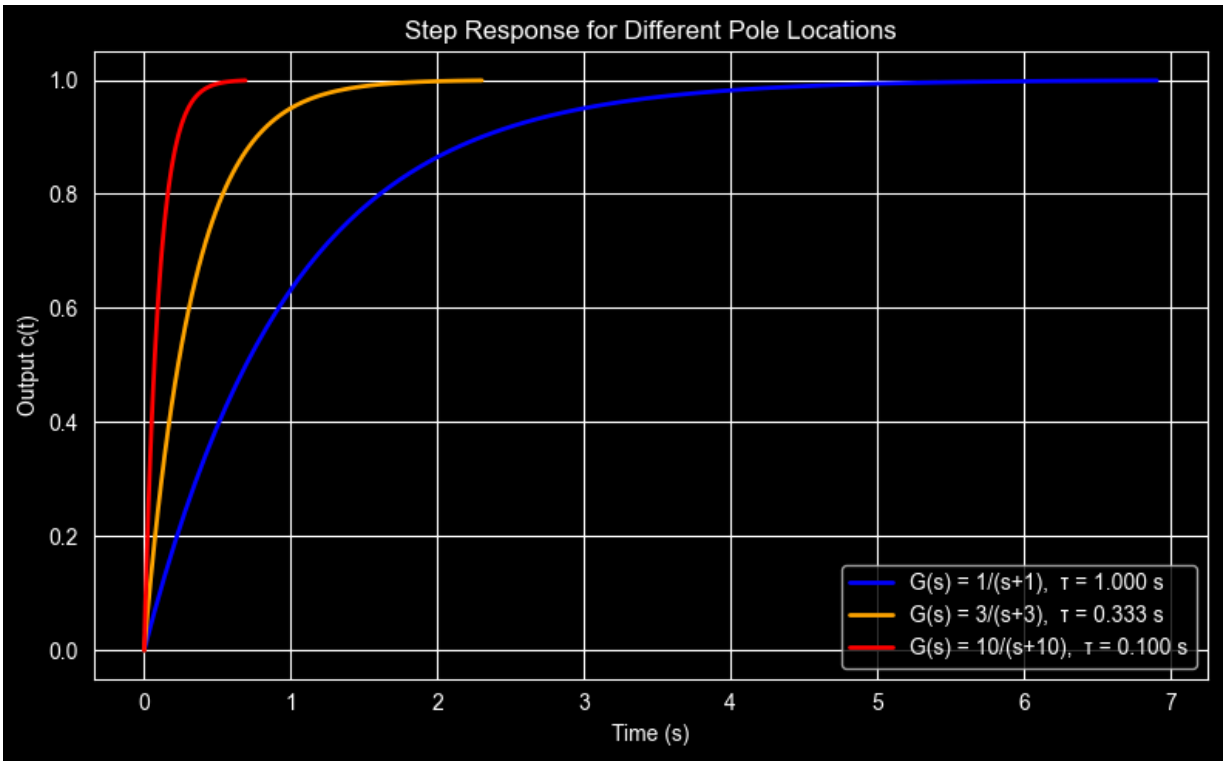


Effect of the Pole Location on Response Speed

The pole location $s = -a$ directly determines how fast the system responds. A pole **farther from the origin** (larger a) produces a **faster** response with a smaller time constant.

We compare three first-order systems with different pole locations.

In [8]:
```
pole_locations = [1, 3, 10]   # values of 'a'
colors = ['blue', 'orange', 'red']

plt.figure(figsize=(8, 5))

for a_val, color in zip(pole_locations, colors):
    G = control.tf([a_val], [1, a_val])
    y, t = control.step(G)
    plt.plot(t, y, color=color, linewidth=2,
             label=f'G(s) = {a_val}/(s+{a_val}),  τ = {1/a_val:.3f} s')

plt.xlabel('Time (s)')
plt.ylabel('Output c(t)')
plt.title('Step Response for Different Pole Locations')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print('\nSummary Table:')
print(f'{"Pole (a)":>10} {"τ (s)":>10} {"Tr (s)":>10} {"Ts (s)":>10}')
print('-' * 45)
for a_val in pole_locations:
    tau = 1 / a_val
    print(f'{a_val:>10}  {tau:>9.4f}  {2.2*tau:>9.4f}  {4.0*tau:>9.4f}')
```



```
Summary Table:
  Pole (a)      τ (s)     Tr (s)     Ts (s)
-------------------------------------------
         1     1.0000     2.2000     4.0000
         3     0.3333     0.7333     1.3333
        10     0.1000     0.2200     0.4000
```

RC Circuit as a First-Order System

A series RC circuit is a classic example of a first-order system. The transfer function from input voltage $V_i(s)$ to capacitor voltage $V_C(s)$ is:

$$G(s) = \frac{V_C(s)}{V_i(s)} = \frac{1}{RCs + 1} = \frac{1/RC}{s + 1/RC}$$

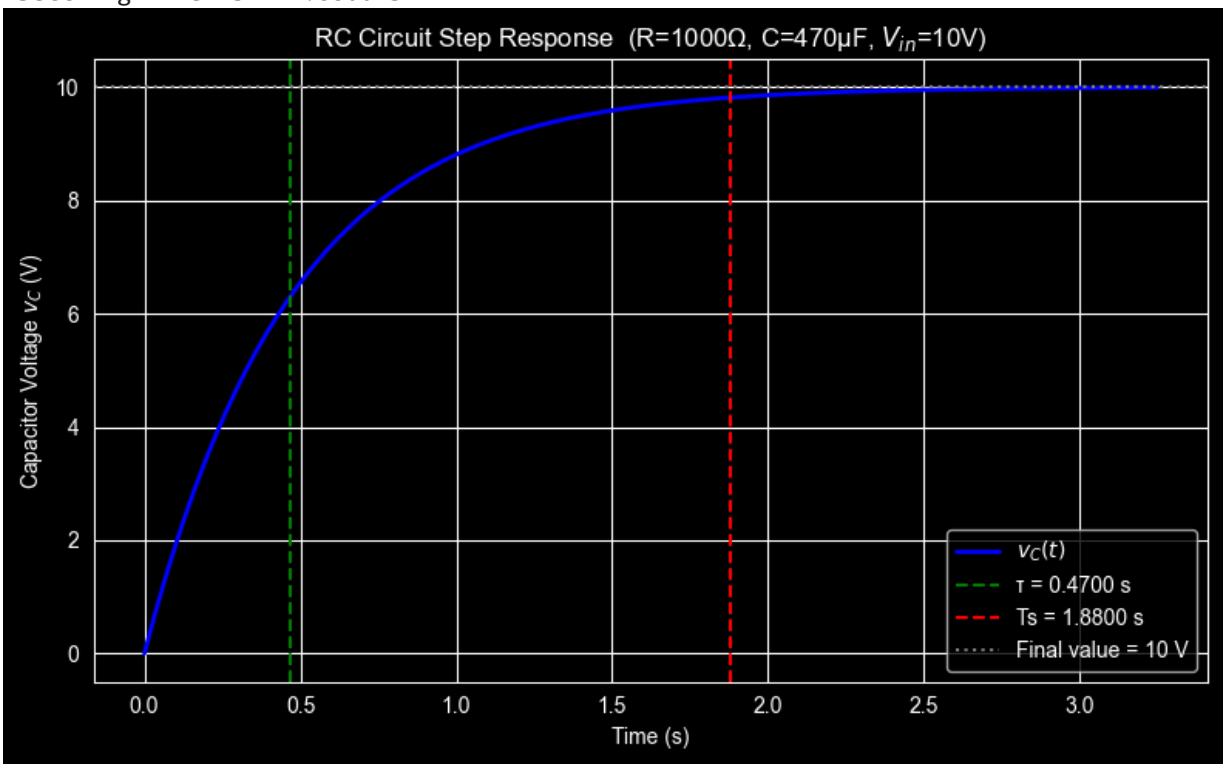
where the time constant is $\tau = RC$.

Consider a circuit with $R = 1\,\text{k}\Omega$ and $C = 470\,\mu\text{F}$, driven by a 10 V step input.

In [9]:
```
R = 1000        # Ohms  (1 kΩ)
C = 470e-6      # Farads (470 µF)
V_in = 10       # Volts  (step amplitude)

tau_RC = R * C
a_RC   = 1 / tau_RC

print(f'RC Time Constant τ = RC = {tau_RC:.4f} s')
print(f'Pole location a = 1/τ = {a_RC:.4f} rad/s')

# Transfer function: G(s) = (1/RC) / (s + 1/RC)
G_rc = control.tf([a_RC], [1, a_RC])
print('\nRC Transfer Function G(s):')
print(G_rc)

# Step response for V_in amplitude (scale the unit step)
y, t = control.step(G_rc)
v_C = V_in * y

# Time specifications
Tr_RC = 2.2 * tau_RC
Ts_RC = 4.0 * tau_RC
print(f'\nRise Time    Tr = {Tr_RC:.4f} s')
print(f'Settling Time Ts = {Ts_RC:.4f} s')

plt.figure(figsize=(8, 5))
plt.plot(t, v_C, 'b-', linewidth=2, label='$v_C(t)$')
plt.axvline(tau_RC, color='green', linestyle='--', label=f'τ = {tau_RC:.4f} s')
plt.axvline(Ts_RC,  color='red',   linestyle='--', label=f'Ts = {Ts_RC:.4f} s')
plt.axhline(V_in,   color='gray',  linestyle=':', label=f'Final value = {V_in} V')

plt.xlabel('Time (s)')
plt.ylabel('Capacitor Voltage $v_C$ (V)')
plt.title(f'RC Circuit Step Response  (R={R}Ω, C={C*1e6:.0f}µF, $V_{{in}}$={V_in}V)')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

```
RC Time Constant τ = RC = 0.4700 s
Pole location a = 1/τ = 2.1277 rad/s

RC Transfer Function G(s):
<TransferFunction>: sys[15]
Inputs (1): ['u[0]']
Outputs (1): ['y[0]']

    2.128
  ---------
  s + 2.128

Rise Time    Tr = 1.0340 s
Settling Time Ts = 1.8800 s
```



Exercises

Apply what you have learned from the demonstration above to solve the following problems. Add your code in the blank cells provided. Show all computed values and plots.

Exercise 1

Given the transfer function:

$$G(s) = \frac{s+2}{s+7}$$

(a) Identify the **poles** and **zeros** of the system.
(b) Plot the **pole-zero map** on the s-plane.
(c) Compute and plot the **step response** $c(t)$.
(d) Based on the pole location, what is the **time constant** of the natural response?

In [12]:
```
# Exercise 1 — your code here
```

Exercise 2

Given the transfer function:

$$G(s) = \frac{12}{s + 6}$$

(a) Identify the **pole** of the system.
(b) Compute the **time constant** τ, **rise time** T_r, and **settling time** T_s.
(c) Compute and plot the **unit step response** $c(t)$.
(d) Annotate the plot with vertical lines at τ, T_r, and T_s as shown in the demonstration.

In [11]:
```
# Exercise 2 — your code here
```

Exercise 3

An RC circuit has the following component values:

- $R = 2.2\,\text{k}\Omega$
- $C = 100\,\mu\text{F}$

The switch closes at $t = 0$ and the input voltage is a **unit step of 5 V**. Assume **zero initial conditions**.

(a) Determine the **transfer function** $G(s) = V_C(s)/V_i(s)$.
(b) Compute the **time constant** τ, **rise time** T_r, and **settling time** T_s.
(c) Plot the **capacitor voltage** $v_C(t)$ over time.
(d) At what time does $v_C(t)$ first reach **3.16 V**? *(Hint: this is related to one of the time specifications.)*

In [13]:
```
# Exercise 3 — your code here
```

Exercise 4

Three first-order systems are defined as follows:

$$G_A(s) = \frac{5}{s + 5} \qquad G_B(s) = \frac{5}{s + 2} \qquad G_C(s) = \frac{5}{s + 8}$$

(a) For each system, compute the **time constant** τ, **rise time** T_r, and **settling time** T_s.
(b) Plot all three **unit step responses** on the same figure with a legend.

(c) Which system responds **fastest**? Which responds **slowest**? Explain your answer in terms of the **pole locations**.

In [15]:
```
# Exercise 4 — your code here
```

In [ ]:
```

```